

Statistics for Data Science Day 6: Correlation and Covariance



DATA SCIENCE WITH SHIKHAR

APR 22, 2026

Why Should a Data Scientist Care About Correlation?

Imagine you're a data scientist at **Netflix**. Your manager walks up to you and says: *"We've noticed that some users who watch more than 3 hours a day are churning faster. Can you investigate?"*

Your first instinct? You want to find out if there's a **relationship** between hours watched and churn rate. Does watching more content make users more likely to quit? Or less? And how strong is that relationship?

This is exactly where **correlation** comes in. It is the statistical tool that tells you whether two variables are related, how strongly they're related, and in which direction they move together.

As a data scientist, correlation is one of the first things you examine when you get a new dataset. It helps you:

- Understand which features influence your target variable.
 - Detect redundant features (two features that are highly correlated carry the same information and one of them might not be needed).
 - Spot potential multicollinearity issues before building regression models.
 - Generate hypotheses for deeper analysis.
-

What is Correlation?

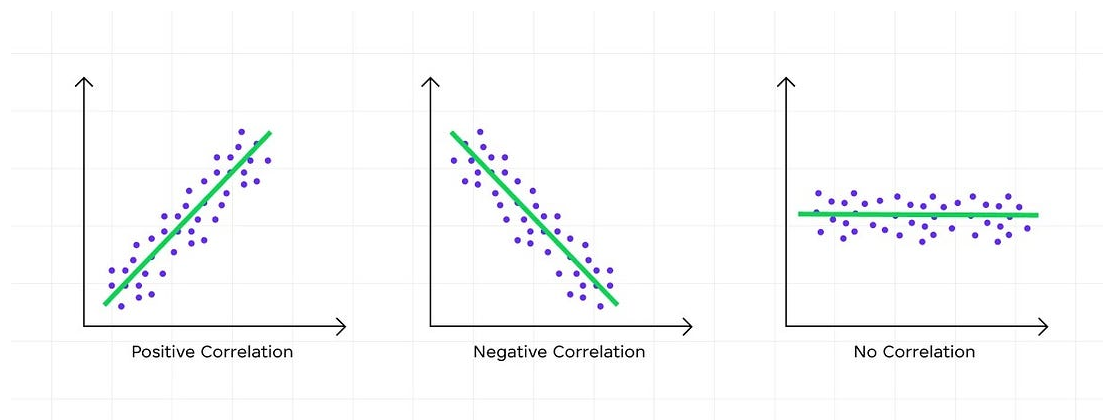
Correlation is a statistical measure that shows the strength and direction of the relationship between two numerical variables.

The key word here is *relationship*. When we say two variables are correlated, we mean that changes in one variable tend to be associated with changes in the other.

Correlation is expressed through a number called the **correlation coefficient**, most often denoted by **r**. This value always falls between **-1 and +1**, and it gives you two pieces of information at once: the **direction** and the **strength** of the relationship.

Think of it as a ruler that goes from -1 on one end to +1 on the other, with 0 in the middle.

Direction of Correlation



Positive Correlation (r close to +1)

When two variables move **in the same direction** wherein as one goes up, the other goes up too, we say they are positively correlated.

Everyday examples:

- Height and weight: Taller people tend to weigh more.
- Temperature and ice cream sales: Hotter days → more ice cream sold.
- Study hours and exam scores: More studying → higher scores.

Big-Tech example: At **Amazon**, the number of product reviews and the product's sales volume are positively correlated. Products with more reviews tend to sell more partly because reviews build trust, and partly because popular products accumulate more reviews. When Amazon's data team notices a drop in review count for a category, it often signals a potential sales dip ahead.

Negative Correlation (r close to -1)

When two variables move in **opposite directions**, as one goes up, the other goes down and we have a negative correlation.

Everyday examples:

- Exercise time and body fat: More exercise → less body fat.
- Price of a product and demand: As price increases, demand decreases.
- Hours of TV watched and exam scores: More TV → lower scores (generally).

Big-Tech example: At **Uber**, there's a well-known negative correlation between driver wait times and user retention. As estimated arrival times (ETA) increase, users are less likely to complete a booking. This insight drove Uber to heavily optimise their driver-matching algorithms to keep ETAs low.

Zero Correlation ($r \approx 0$)

When there is **no systematic relationship** between two variables, the correlation coefficient is close to 0. Knowing the value of one variable tells you nothing useful about the other.

Examples:

- A person's shoe size and their salary.
- The number of films someone watches per year and their blood pressure.
- Daily coffee consumption and IQ score.

How Strong is the Relationship? A Practical Guide

The absolute value of "*r*" tells you how **strong** the correlation is. Here's a simple guide most data scientists use:

The value of <i>r</i>	Strength of Correlation
< 0.1	No correlation
$0.1 < 0.3$	Little correlation
$0.3 < 0.5$	Medium correlation
$0.5 < 0.7$	High correlation
$0.7 < 1$	Very high correlation

Note: These are general guidelines, not hard rules. In social science, an *r* of 0.3 might be meaningful. In physics or engineering, you'd expect $r > 0.95$. Always interpret correlation in the context of your domain.

Types of Correlation Coefficients

How Do We Measure Correlation in Data Science?

The Correlation Coefficient (r)

Understanding the Strength and Direction

Not all data is the same. Depending on your data's nature and distribution, you should choose the right type of correlation coefficient. There are three main ones:

- **Pearson's r** : for linear relationships between continuous, normally distributed data.
- **Spearman's ρ (rho)** : for monotonic relationships, works on ranked data, handles outliers better.
- **Kendall's τ (tau)** : similar to Spearman, but more robust for small samples.

Let's go deep on each one.

Covariance: The Building Block

Before we get to correlation, we need to understand **covariance** because correlation is literally built on top of it.

Covariance measures how two variables change together. If both tend to be above their average at the same time, the covariance is positive.

If one tends to be above when the other is below, it's negative.

The formula for covariance is:

$$\text{Cov}(X, Y) = \Sigma[(X_i - \bar{X})(Y_i - \bar{Y})]/(n - 1)$$

The problem with covariance: It depends on the units of measurement. If you measure height in centimeters vs. meters, you get a completely different covariance value even though the underlying relationship is the same. This makes it hard to compare covariances across different datasets.

This is exactly why we need **correlation**: it standardizes covariance by dividing by the standard deviations of both variables, giving us a unit-free value always between -1 and +1.

```
import numpy as np
import pandas as pd

# Example: Hours studied vs. Exam Score
hours_studied = [1, 2, 3, 4, 5, 6, 7, 8]
exam_score = [45, 50, 55, 60, 65, 70, 75, 80]

# Covariance
cov_matrix = np.cov(hours_studied, exam_score)
print("Covariance Matrix:")
print(cov_matrix)

# The Cov(X, Y) value
cov_xy = cov_matrix[0][1]
print(f"\nCovariance between Hours Studied and Exam Score: {cov_xy:.2f}")
```

Covariance Matrix:

[[6. 21.]

[21. 126.]]

Covariance between Hours Studied and Exam Score: 21.00

The covariance is 21 but is that high or low? We have no way to tell without context. That's why we divide it by the standard deviations to get correlation.

Pearson Correlation Coefficient (r)

The **Pearson correlation coefficient** is the most widely used measure of correlation. It measures the strength and direction of the **linear** relationship between two continuous variables.

The formula is:

$$r = Cov(X, Y) / (\sigma_X \times \sigma_Y)$$

$$r = \Sigma[(X_i - \bar{X})(Y_i - \bar{Y})] / \sqrt{[\Sigma(X_i - \bar{X})^2 \times \Sigma(Y_i - \bar{Y})^2]}$$

By dividing covariance by the product of standard deviations, we normalize the value to always be between -1 and +1, regardless of the original units.

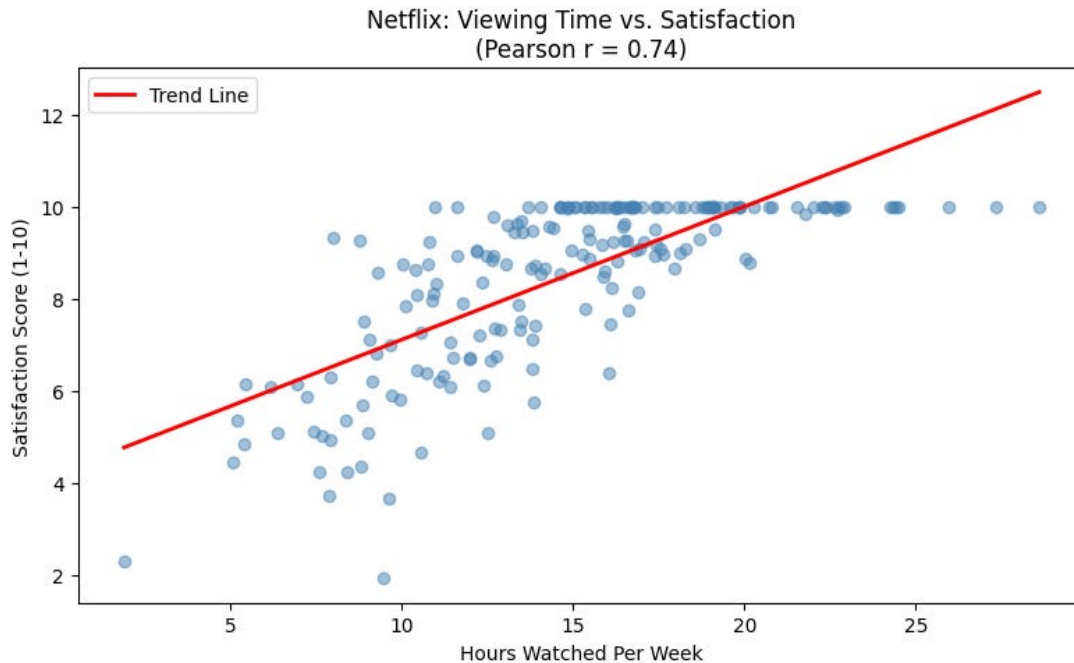
Interpreting the Sign and Magnitude

- **r = +1** → Perfect positive linear relationship.
- **r = -1** → Perfect negative linear relationship.
- **r = 0** → No linear relationship.
- **r = 0.85** → Strong positive linear relationship.

- $r = -0.60$ → Moderate negative linear relationship.

Python Example: Netflix Viewing Time vs. User Satisfaction

Let's say we're analyzing whether the total hours a Netflix user watches per week is correlated with their satisfaction score (rated 1–10 in post-viewing surveys).



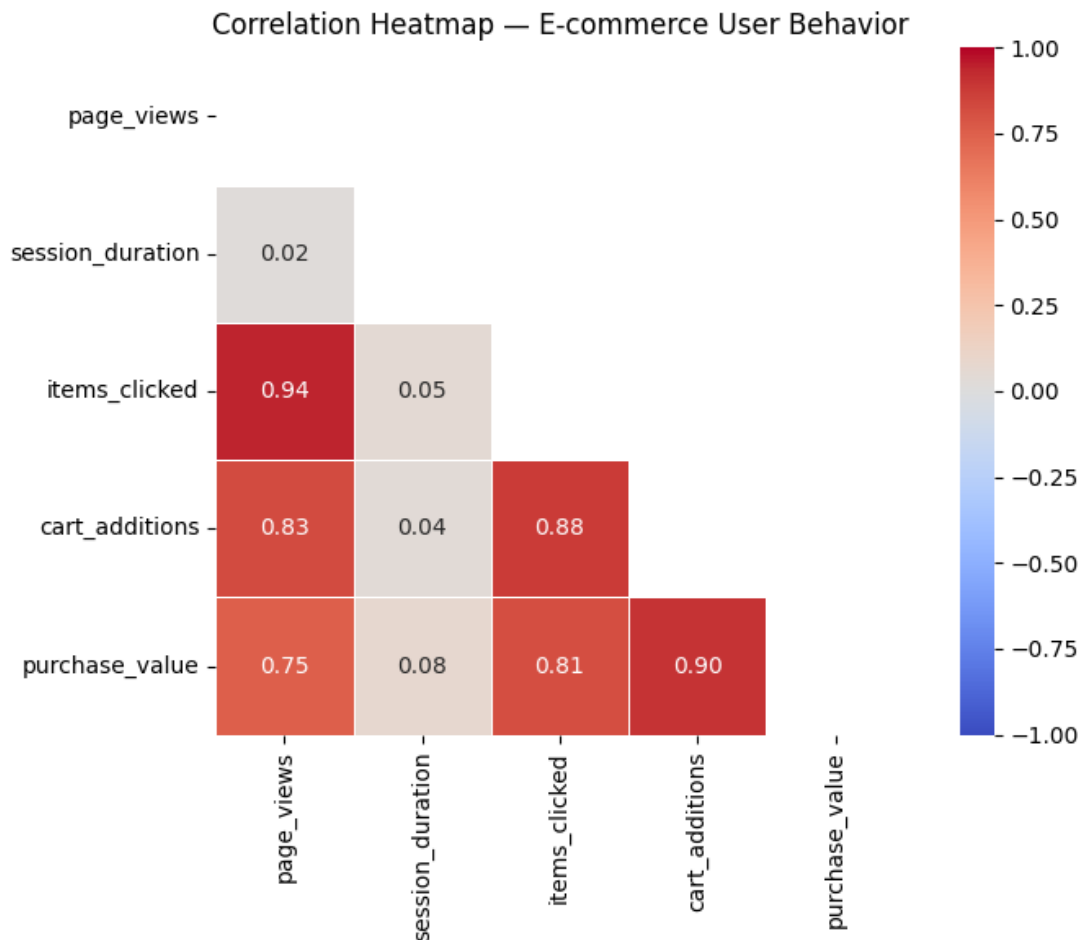
Note : See the Python code to execute this yourself [here](#).

Always check whether the relationship actually looks linear before trusting Pearson's r . You can do this by plotting a scatter plot.

Python Example: Correlation Matrix & Heatmap (The Real Power Tool)

In practice, you rarely look at just two variables. You examine many features at once using a **correlation matrix**, a table showing the correlation between every pair of variables.

Here's how data scientists at companies like **Google** or **Meta** use it during Exploratory Data Analysis (EDA):



Note: See the Python code to execute it yourself [here](#)

Reading a heatmap is simple:

- **Red cells (close to +1):** Strong positive correlation.
- **Blue cells (close to -1):** Strong negative correlation.
- **White/light cells (close to 0):** Little to no linear relationship.

In this example, you'd see that `cart_additions` and `purchase_value` are highly correlated which makes total sense. This is the kind of insight that helps

a product team decide which features to prioritize in a checkout flow redesign.

Assumptions of Pearson Correlation

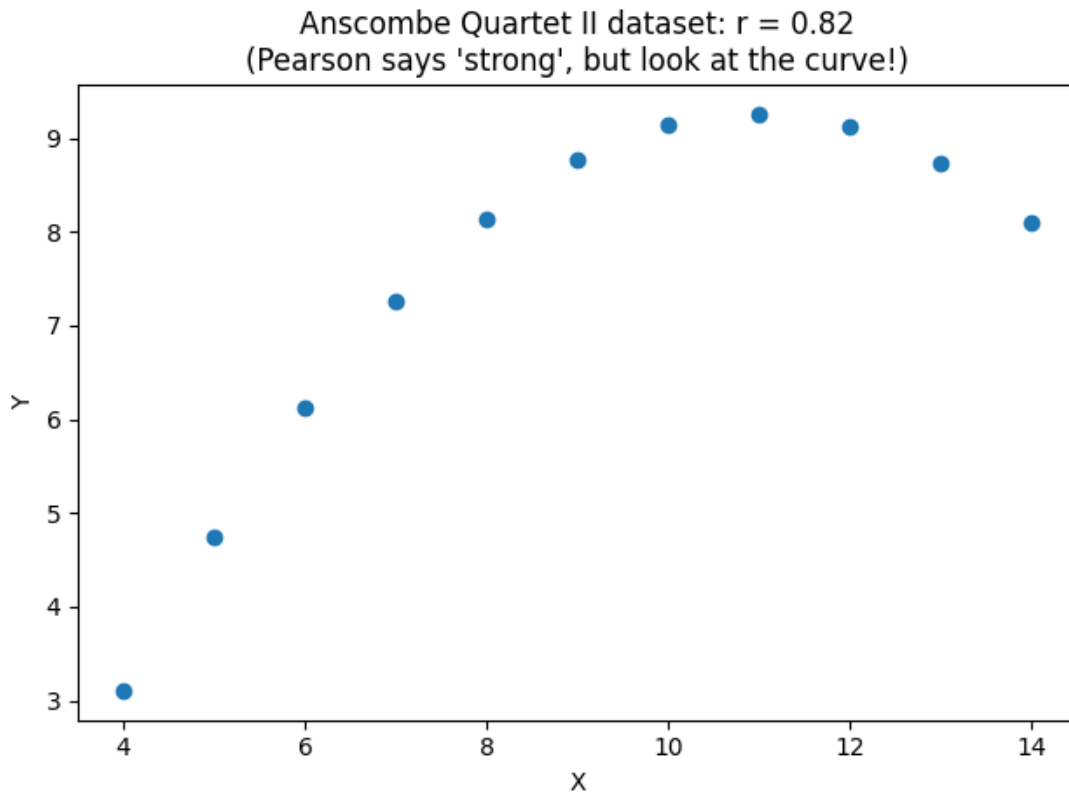
Pearson's r is a **parametric** measure, meaning it makes certain assumptions about your data. Violating these can give you misleading results:

1. **Linearity** : The relationship between the two variables must be linear.
2. **Normality** : Both variables should be approximately normally distributed.
3. **No significant outliers** : Outliers can drag the correlation coefficient towards them, making the relationship look stronger or weaker than it really is.
4. **Continuous data** : Both variables should be continuous (not categorical or ordinal).

Disadvantages of Pearson Correlation

1. Blind to Non-Linear Relationships

Consider the relationship between stress levels and job performance. Up to a moderate level, more stress can push someone to perform better (positive correlation). But beyond a threshold, high stress destroys performance (negative correlation). This U-shaped relationship will be ignored by Pearson, as it will focus on the overall relationship which is mostly linear hence completely missing the real pattern.



Note: See the Python code for the same [here](#)

Lesson: Always visualize your data. Never trust a correlation number without a scatter plot.

2. Sensitivity to Outliers

```
import numpy as np
from scipy import stats

# Normal data
x_normal = [2, 4, 5, 6, 8, 9, 10, 12]
y_normal = [3, 5, 4, 7, 9, 8, 11, 13]

# Same data with one extreme outlier
x_outlier = x_normal + [50]
y_outlier = y_normal + [2]
```

```
r_normal, _ = stats.pearsonr(x_normal, y_normal)
r_outlier, _ = stats.pearsonr(x_outlier, y_outlier)

print(f"Pearson 'r' without outlier: {r_normal:.4f}")
print(f"Pearson 'r' WITH outlier: {r_outlier:.4f}")
# The single outlier can dramatically shift the r value
```

Pearson 'r' without outlier: 0.9636

Pearson 'r' WITH outlier: -0.3026

This is exactly why Spearman's correlation exists, it's much more resistant to outliers.

Spearman Rank Correlation (ρ)

Spearman's rank correlation is the go-to alternative when Pearson's assumptions are violated. Instead of working with raw data values, it first converts them into **ranks** and then computes Pearson's r on those ranks.

This makes it:

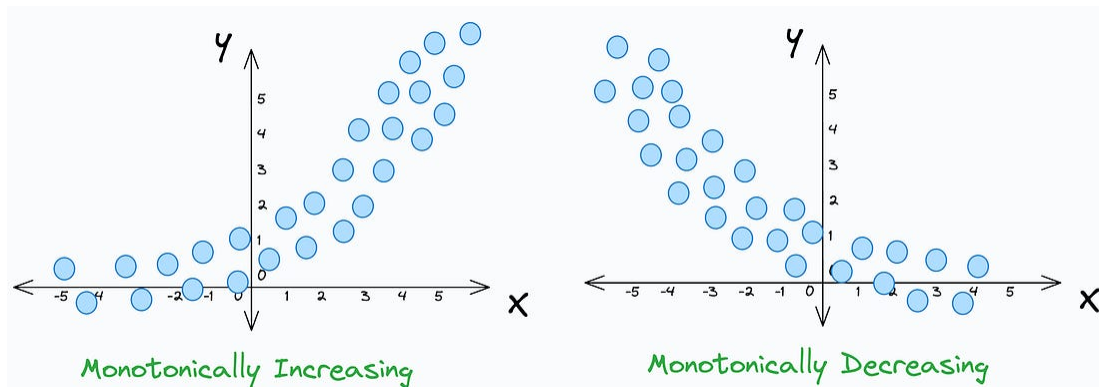
- **Non-parametric** : No assumption of normal distribution.
- **Robust to outliers** : Because ranks aren't influenced by extreme values.
- **Suitable for ordinal data** : Data that has an order but not a fixed scale (e.g., movie ratings: 1 star, 2 stars... 5 stars).

Understanding Monotonic Relationships

While Pearson needs the relationship to be **linear**, Spearman only needs it to be **monotonic**.

A monotonic relationship means: as X increases, Y either *always increases* or *always decreases* but not necessarily in a straight line.

Think of it like climbing a mountain on a winding road. You keep going up (monotonically increasing), but the incline is not constant. Spearman would perfectly capture this. Pearson would struggle.



Monotonically Increasing: As one variable increases, the other never decreases. Example: age and number of years of work experience.

Monotonically Decreasing: As one variable increases, the other never increases. Example: time spent in a customer service call and customer satisfaction score.

Not Monotonic: As one variable increases, the other sometimes goes up and sometimes down. Example: caffeine intake and focus wherein low caffeine helps focus, moderate caffeine helps more, but too much caffeine kills focus.

How Spearman's Works : Step by Step

Given data for X and Y:

1. Rank the values of X (1 = smallest). In case of ties, assign the average rank.
2. Rank the values of Y similarly.

3. Apply Pearson's correlation formula to the **ranks**, not the raw values.

The shortcut formula (when there are no ties) is:

$$\rho = 1 - [6 \times \sum d_i^2] / [n(n^2 - 1)]$$

Where d_i is the difference in ranks for each pair, and n is the number of observations.

Python Example: App Store Ratings (Google Play)

Imagine you're at **Google** analyzing the Play Store. You want to know if there's a relationship between an app's user engagement score (a custom internal metric, not normally distributed, has some extreme values) and the number of active installs.

```
import pandas as pd
import numpy as np
from scipy import stats

# Simulated Google Play app data
np.random.seed(7)
n = 50

# Engagement score is right-skewed (not normal) — some apps are viral
engagement_score = np.random.exponential(scale=20, size=n)

# Active installs: loosely correlated, but with some wild outliers
active_installs = engagement_score * 50000 + np.random.normal(0, 500000, n)
active_installs = np.abs(active_installs)

df = pd.DataFrame({
    'engagement_score': engagement_score,
```

```

    'active_installs': active_installs
})

# --- Pearson (not ideal here, data isn't normal) ---
pearson_r, pearson_p = stats.pearsonr(df['engagement_score'],
df['active_installs'])

# --- Spearman (better choice) ---
spearman_r, spearman_p = stats.spearmanr(df['engagement_score'],
df['active_installs'])

print("--- Pearson ---")
print(f"r = {pearson_r:.4f}, p-value = {pearson_p:.4f}")

print("\n--- Spearman ---")
print(f"p = {spearman_r:.4f}, p-value = {spearman_p:.4f}")

# Show ranks
df['engagement_rank'] = df['engagement_score'].rank()
df['installs_rank'] = df['active_installs'].rank()
print("\nFirst 5 rows with ranks:")
print(df.head())

```

Real-World Example: Netflix Star Rating → Thumbs Up/Down Migration

Back in 2017, Netflix switched from 5-star ratings to a thumbs up/thumbs down system. The reason? Their data scientists found that the 5-star numerical rating was **ordinal** data where users didn't experience the gap between 3 and 4 stars the same way they experienced the gap between 1 and 2 stars. So to measure correlations involving ratings, **Spearman's was the right tool**, not Pearson's.

```

# Before Netflix's switch: 5-star rating vs. re-watch probability
# Star rating is ordinal (1-5), re-watch is continuous (0-100%)

```

```
star_ratings = [1, 2, 2, 3, 3, 4, 4, 4, 5, 5]
rewatch_prob = [5, 12, 18, 35, 40, 60, 65, 70, 85, 92]

pearson_r_ = stats.pearsonr(star_ratings, rewatch_prob)
spearman_r_ = stats.spearmanr(star_ratings, rewatch_prob)

print(f"Pearson r: {pearson_r_:4f} ← Assumes equal gap between stars (wrong!)")
print(f"Spearman p: {spearman_r_:4f} ← Only cares about rank order (correct!)")
```

When to Use Spearman Over Pearson?

Use Spearman when:

- Your data is **ordinal** (ratings, rankings, satisfaction levels).
- Your data has **significant outliers**.
- Your data is **not normally distributed** (e.g., income data, which is heavily right-skewed).
- The relationship looks **curved but monotonic** on a scatter plot.

Kendall's Tau (τ) : The Underrated One

Kendall's tau is another rank-based correlation method, very similar to Spearman's. The key difference is in how it counts agreement between pairs.

For every pair of observations (i, j), Kendall's tau checks: do X and Y move in the same direction? If yes, it's a **concordant pair**. If they move in opposite directions, it's a **discordant pair**.

$$\tau = (\text{Concordant Pairs} - \text{Discordant Pairs}) / \text{Total Pairs}$$

When to prefer Kendall's τ :

- When your **sample size is small** : Kendall's is more accurate with fewer data points.
- When you have **many tied ranks** in your data.
- It's also considered more interpretable: a τ of 0.6 means 60% more concordant pairs than discordant pairs.

```
from scipy import stats
import numpy as np

# Small sample: Customer satisfaction rank vs. Support response time rank
# (Both are ranked 1 = best, 5 = worst)

satisfaction_rank = [1, 2, 3, 4, 5]
response_time_rank = [2, 1, 3, 5, 4]

tau, p_value = stats.kendalltau(satisfaction_rank, response_time_rank)
print(f"Kendall's  $\tau$ : {tau:.4f}")
print(f"P-value: {p_value:.4f}")
```

Big-Tech Use Case: At **Microsoft**, when evaluating search result quality, engineers often rank pages by relevance and compare rankings from different ranking algorithms using Kendall's tau. A high τ means two algorithms agree on what's relevant where a low one means they're giving very different answers and someone needs to investigate.

Choosing the Right Correlation Method

Here's a simple decision guide:

Is your data continuous and normally distributed?

→ YES → Use Pearson's r

→ NO ↓

Is your data ordinal OR does it have outliers OR is the relationship non-linear but monotonic?

→ YES → Use Spearman's ρ

Is your sample very small ($n < 30$) OR do you have many tied ranks?

→ YES → Use Kendall's τ

The Most Important Lesson: Correlation $\neq$ Causation

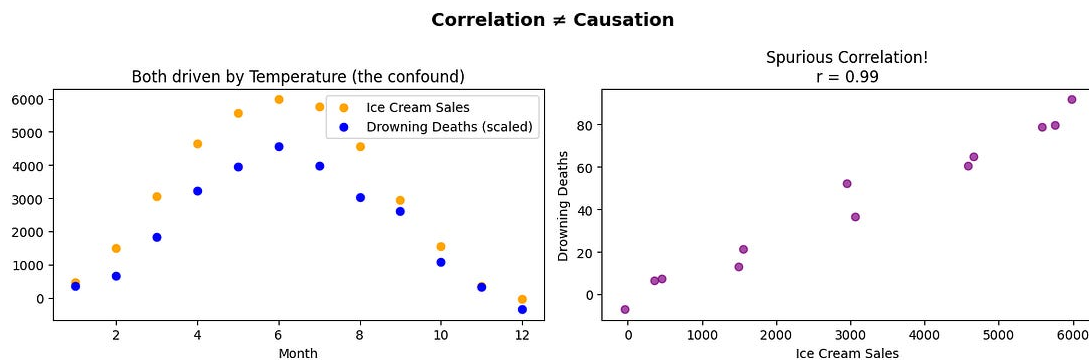
This is perhaps the most critical concept in all of statistics. **Just because two variables are correlated does not mean one causes the other.**

There's a famous example that Nicholas Cage movie releases are correlated with drowning deaths where both peak in the summer. Obviously, Cage's films don't cause drowning. They're both driven by a third variable: summer (when both pool activity and movie releases peak). This third variable is called a **confounding variable** or **lurking variable**.

Big-Tech example: Early studies showed that Facebook users who shared more posts had higher engagement with ads. A product team might jump to: "Let's make people share more posts to increase ad revenue!" But the real relationship is reversed, as people who are more engaged with Facebook in general, are found to share more *and* click

more ads. Forcing sharing without genuine engagement doesn't improve ad performance.

Always ask: Is there a third variable that could explain this correlation? Could the causality run in the opposite direction?



Note: See the Python code for the same [here](#)

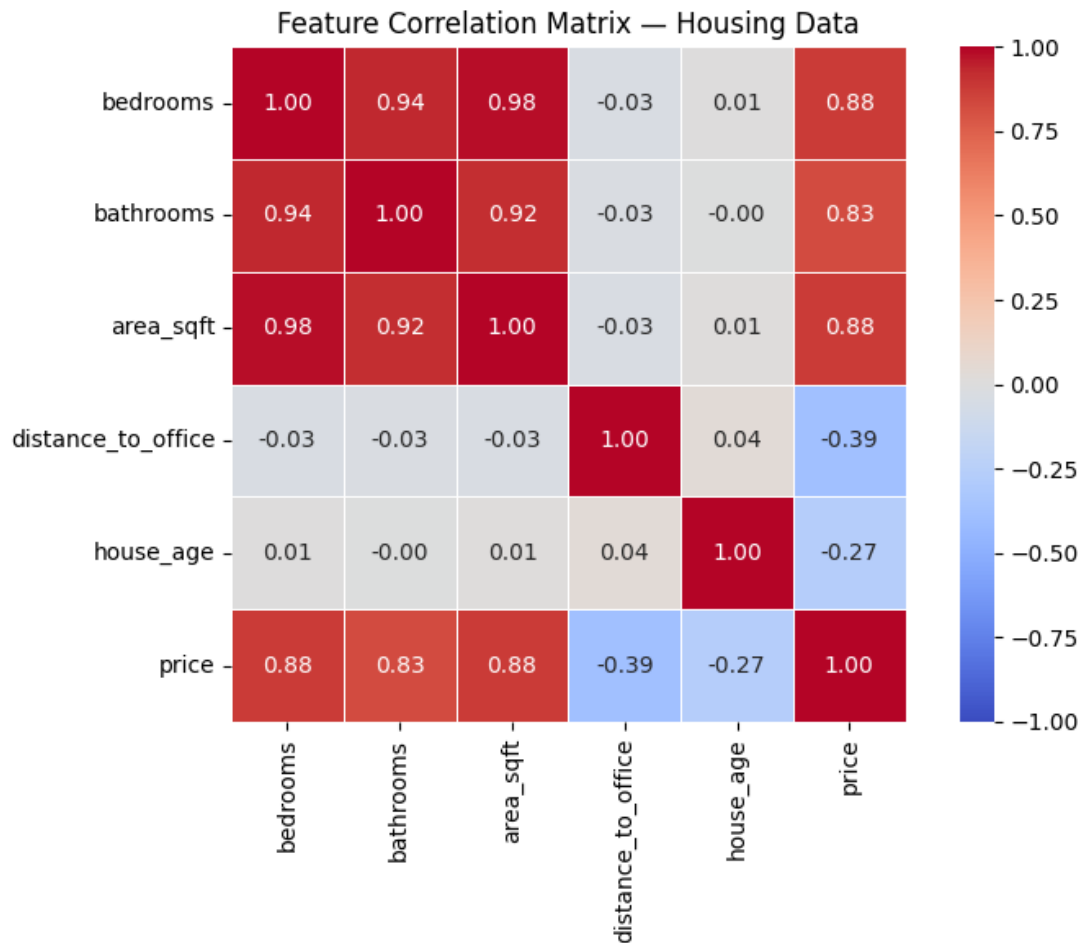
You can see lots of fun examples for the same on this website :

<https://www.tylervigen.com/spurious-correlations>

Practical Use Case: Correlation for Feature Selection in ML

One of the most practical applications of correlation in data science is **feature selection** for deciding which variables to include in your machine learning model.

The idea: If two input features (predictors) are very highly correlated with each other (say, $r > 0.90$), they're carrying almost identical information. Including both doesn't help the model instead it can actually hurt it by causing **multicollinearity** in linear models.



Note: See the Python code for the same [here](#)

In this example, bedrooms and bathrooms are highly correlated (both represent the size of the house). So you'd drop one of them before feeding data into a linear regression model.

Full Workflow: From Data to Insights

Here's how a professional data scientist would approach correlation analysis end-to-end:

Important Note: Before going through the code below, read a little about Hypothesis Testing as there are some terms from that topic.

```

import pandas as pd
import numpy as np
from scipy import stats
import seaborn as sns
import matplotlib.pyplot as plt

def correlation_analysis(df, target_col, method='auto'):
    """
    Full correlation analysis workflow.

    Args:
        df: DataFrame with numeric columns
        target_col: The column you want to correlate others against
        method: 'pearson', 'spearman', or 'auto' (auto chooses based on normality)
    """
    numeric_df = df.select_dtypes(include=[np.number])

    # Step 1: Check normality using Shapiro-Wilk test
    if method == 'auto':
        _, p_val = stats.shapiro(df[target_col])
        method = 'pearson' if p_val > 0.05 else 'spearman'
        print(f"Normality test p-value: {p_val:.4f}")
        print(f"Auto-selected method: {method.upper()}\n")

    # Step 2: Compute correlations with the target
    correlations = {}
    for col in numeric_df.columns:
        if col == target_col:
            continue
        if method == 'pearson':
            r, p = stats.pearsonr(df[target_col], df[col])
        else:
            r, p = stats.spearmanr(df[target_col], df[col])
        correlations[col] = {'correlation': r, 'p_value': p, 'significant': p < 0.05}

```

```

result_df = pd.DataFrame(correlations).T
result_df = result_df.sort_values('correlation', key=abs, ascending=False)

# Step 3: Display results
print(f"Correlation with '{target_col}':\n")
print(result_df.round(4).to_string())

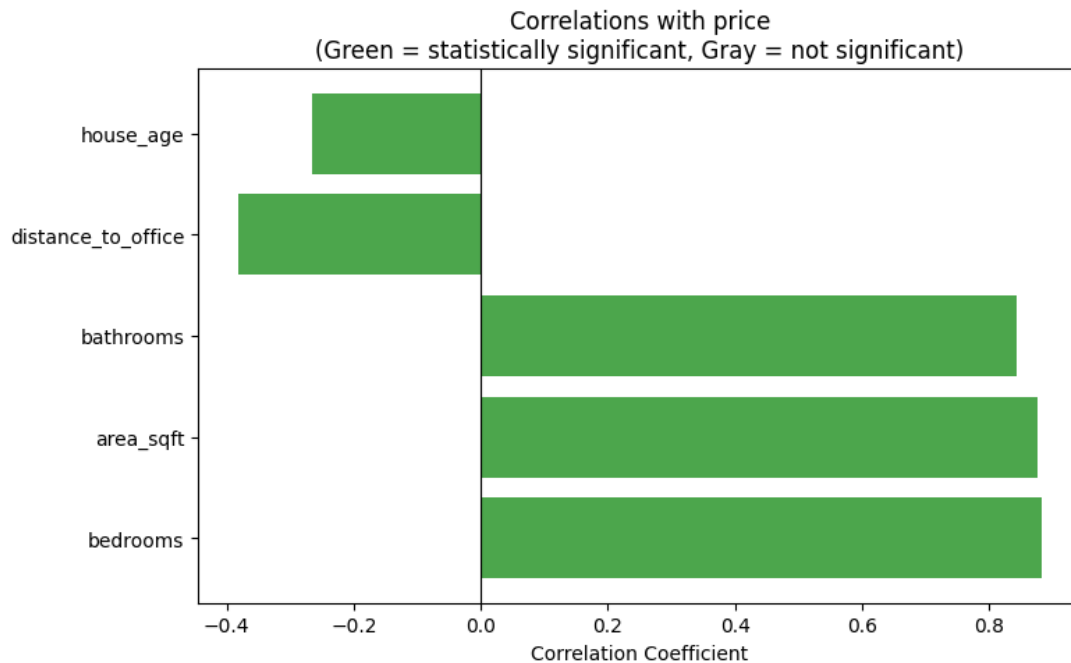
# Step 4: Visualize
plt.figure(figsize=(8, 5))
colors = ['green' if v else 'gray' for v in result_df['significant']]
bars = plt.barh(result_df.index, result_df['correlation'], color=colors, alpha=0.7)
plt.axvline(0, color='black', linewidth=0.8)
plt.xlabel('Correlation Coefficient')
plt.title(f'Correlations with {target_col}\n(Green = statistically significant, Gray = not significant)')
plt.tight_layout()
plt.savefig('correlation_bar_chart.png', dpi=150)
plt.show()

return result_df

# Usage
correlation_analysis(df, target_col='price', method='auto')

```

	correlation	p_value	significant
bedrooms	0.882833	0.0	True
area_sqft	0.875662	0.0	True
bathrooms	0.843758	0.0	True
distance_to_office	-0.381699	0.0	True
house_age	-0.265601	0.000003	True



Correlation is one of the most powerful and most misused tools in data science. Used correctly with the right method, combined with visualization, and with a healthy skepticism about causation, it can unlock deep insights from your data.

Thanks for reading DSAI with Shikhar!
Subscribe for free to receive new posts and
support my work.

✓ Subscribed